

CG 作业 1 报告

李惟乐 PB23050929 2025.03.02

1 椭圆绘制

椭圆类的元素和函数接口和矩形是一致的，在确定椭圆角度（默认为 0.0）后，椭圆就只有两个自由度，通过首点和末点来确定唯一的椭圆。椭圆中心坐标为首末两点中点，长轴分别为 x 和 y 方向坐标绝对差。具体参数设定如下：

```
draw_list->AddEllipse(  
    ImVec2(  
        config.bias[0] + (start_point_x_ + end_point_x_) / 2, config.bias[1] + (start_point_y_ + end_point_y_) / 2),  
    ImVec2( std::fabs(start_point_x_ - end_point_x_) / 2, std::fabs(start_point_y_ - end_point_y_) / 2 ),  
    IM_COL32(  
        config.line_color[0],  
        config.line_color[1],  
        config.line_color[2],  
        config.line_color[3]),  
    0.f, // No rotation  
    ImDrawFlags_None,  
    config.line_thickness);
```

2 多边形绘制

通过记录一系列点的坐标来表示多边形，因此元素首先包括两个 `vector` 容器；其次为了反映鼠标实时位置变化，增添 `real_time` 元素，记录当前鼠标所在位置；最后为一个 `bool` 元素 `is_closed`，用于判断多边形对象当前是否闭合。函数接口多出两个，其一是给多边形添加新的点，其二为令多边形闭合。具体元素构成和函数接口定义如下：

 Polygon
• <code>Polygon() = default : void</code> • <code>Polygon(std::vector<float> x_list, std::vector<float> y_list, bool is_closed_ = false) : void</code> • <code>~Polygon() constexpr = default : void</code>
• <code>add_control_point(float x, float y) : void</code> • <code>draw(const Config & config) const : void</code> • <code>set_closed() : void</code> • <code>update(float x, float y) : void</code>
□ <code>is_closed : bool</code> □ <code>real_time_x : float</code> □ <code>real_time_y : float</code> □ <code>x_list_ : std::vector<float></code> □ <code>y_list_ : std::vector<float></code>

多边形绘制逻辑为左键添加节点，右键令图形闭合，绘制完成。绘制多边形过程采用多次调用绘制直线实现，并且区分多边形是否闭合。

为此修改鼠标左键点击逻辑 `mouse_click_event()`，新增鼠标右键点击逻辑 `mouse_right_click_event()`。具体如下：

鼠标左键点击逻辑：若当前未处于绘制模式，则进入绘制模式，创建多边形对象。若

当前已经处于绘制模式，而且绘制对象为多边形，则获取当前鼠标位置，作为新增节点添加进多边形。

鼠标右键点击逻辑：若当前处于绘制模式，而且绘制对象为多边形，则令多边形闭合，退出绘制模式。

3 自由绘制

自由绘制逻辑与多边形绘制相似，都是通过多次调用直线实现。自由绘制仅需通过两个向量记录一系列二维点坐标，因此定义较为简单，具体如下：

C FreeHand	
●	FreeHand() = default : void
●	FreeHand(std::vector<float> x_list, std::vector<float> y_list) : void
●	~FreeHand() constexpr = default : void
●	add_control_point(float x, float y) : void
●	draw(const Config & config) const : void
●	update(float x, float y) : void
□	x_list_ : std::vector<float>
□	y_list_ : std::vector<float>

绘制逻辑与多边形稍有不同：鼠标左键按下时进入绘制模式，在鼠标按下并移动的过程中不断向自由绘制对象中增添鼠标当前所在点，直到鼠标左键释放。为此修改鼠标移动事件 `mouse_move_event()`，鼠标释放事件 `mouse_release_event()`。

鼠标移动事件新增一个判断：如果当前处于绘制状态，而且绘制对象为自由绘制，则向绘制对象添加当前鼠标所在点；

鼠标释放事件：如果当前处于绘制模式，且绘制对象为自由绘制，则向 `shape_list_` 压入自由绘制对象，退出绘制模式。

4 总体逻辑：

实验总体是针对画布这一对象进行操作，鼠标所画的图形放在 `shape_list_` 中，时刻对 `shape_list_` 中的对象进行重绘，总体类图见下一页：

